

Python Error Handling

A practical guide to try / except / else / finally, raising exceptions, and building your own exception types — interview-ready notes with fully commented code.

An **exception** is an event that interrupts the normal flow of a program when an error occurs. Instead of letting the program crash, Python lets you *catch* exceptions and decide what to do about them. The core tool for this is the try / except block.

1. The Basics: try / except

Code that might fail goes inside try. If an exception occurs, Python immediately jumps to the matching except block instead of crashing the program.

```
# --- Basic try/except ---

try:
    # Code that MIGHT raise an exception goes here.
    number = int(input("Enter a number: "))
    result = 10 / number

except ZeroDivisionError:
    # This block runs ONLY if a ZeroDivisionError was raised above.
    print("You can't divide by zero!")

except ValueError:
    # This block runs ONLY if int() failed to convert the input.
    print("That wasn't a valid number.")

# If no exception occurs, both except blocks are skipped entirely,
# and execution just continues normally after the try/except.
```

Note: Python checks except blocks top to bottom and stops at the first one that matches. Order matters: put more specific exceptions before more general ones.

2. Common Built-in Exceptions

Exception	Raised when...
ValueError	A value has the right type but an invalid value (e.g. int("abc"))
TypeError	An operation is applied to an object of the wrong type (e.g. "2" + 2)
KeyError	A dictionary key doesn't exist
IndexError	A list/sequence index is out of range
ZeroDivisionError	Dividing (or modulo) by zero
FileNotFoundError	Trying to open a file that doesn't exist

AttributeError	Accessing an attribute/method that doesn't exist on an object
ImportError	A module or name fails to import

3. else and finally

`else` runs only if the `try` block succeeded with **no** exception. `finally` always runs — whether an exception happened or not — and is the right place for cleanup code like closing files or connections.

```
# --- Full try / except / else / finally ---

def read_config(path):
    try:
        f = open(path, "r", encoding="utf-8")
        data = f.read()

    except FileNotFoundError:
        # Runs only if open() failed because the file doesn't exist.
        print(f"Config file '{path}' not found, using defaults.")
        return {}

    else:
        # Runs ONLY if the try block raised NO exception at all.
        # Good place for code that should run after a clean success.
        print("Config loaded successfully.")
        return {"raw": data}

    finally:
        # Runs NO MATTER WHAT -- success, exception, or even a return
        # statement above. Perfect for guaranteed cleanup.
        try:
            f.close()
        except NameError:
            # f was never created because open() failed --
            # nothing to close, so just ignore this.
            pass
        print("read_config() finished.")
```

4. Catching Multiple Exceptions

```
# --- Catching several exception types together ---

try:
    value = my_dict["missing_key"]
except (KeyError, TypeError) as e:
    # A tuple of exception types catches ANY of them with one block.
    # 'as e' gives you the actual exception object.
    print(f"Lookup failed: {e}")

# --- Catching "everything" (use sparingly!) ---
try:
    risky_operation()
except Exception as e:
    # Exception is the base class for almost all built-in exceptions
    # (it excludes things like SystemExit and KeyboardInterrupt).
    # This is a broad net -- fine for logging/top-level handlers,
    # but bad practice if it hides bugs you should actually be fixing.
    print(f"Something went wrong: {type(e).__name__}: {e}")
```

Note: Avoid a bare 'except:' with no exception type. It also catches SystemExit and KeyboardInterrupt, which can make programs impossible to stop with Ctrl+C. Use 'except Exception:' instead if you truly need a catch-all.

5. Raising Exceptions Yourself

Use `raise` to signal that something has gone wrong in your own code. This is how you enforce rules and invariants explicitly rather than letting bad data spread silently.

```
# --- Raising a built-in exception ---

def set_age(age):
    if age < 0:
        # raise stops execution immediately and hands the exception
        # up to whichever try/except (if any) is waiting for it.
        raise ValueError(f"Age cannot be negative, got {age}")
    return age

set_age(-5)  # raises: ValueError: Age cannot be negative, got -5

# --- Re-raising after partially handling an exception ---
def process(data):
    try:
        return 100 / data
    except ZeroDivisionError:
        print("Logging the error before passing it along...")
        raise  # bare 'raise' re-raises the SAME exception,
              # preserving its original traceback

# --- Chaining exceptions with 'from' ---
def load_settings(path):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError as original_error:
        # 'raise ... from original_error' links the two exceptions,
        # so the traceback shows the real root cause, not just this new one.
        raise RuntimeError("Could not load settings") from original_error
```

6. Custom Exception Classes

For anything beyond the simplest scripts, define your own exception classes. They make error handling self-documenting: catching `InsufficientFundsError` is far clearer than catching a generic `ValueError`.

```

# --- Defining and using a custom exception hierarchy ---

class BankError(Exception):
    """Base class for every error this banking module can raise.

    Having one common base class lets callers catch ALL bank-related
    errors with a single 'except BankError' if they want to.
    """
    pass

class InsufficientFundsError(BankError):
    """Raised when a withdrawal exceeds the available balance."""

    def __init__(self, balance, amount):
        # Store extra context on the exception object itself so the
        # caller can inspect exactly what went wrong.
        self.balance = balance
        self.amount = amount
        message = f"Cannot withdraw {amount}: balance is only {balance}"
        super().__init__(message)  # sets up the normal str(e) message

class AccountFrozenError(BankError):
    """Raised when an operation is attempted on a frozen account."""
    pass

class Account:
    def __init__(self, balance):
        self.balance = balance
        self.frozen = False

    def withdraw(self, amount):
        if self.frozen:
            raise AccountFrozenError("This account is frozen.")
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount
        return self.balance

```

```

# --- Using the custom exceptions ---
account = Account(balance=100)

try:
    account.withdraw(250)
except InsufficientFundsError as e:
    # We can access the custom attributes we stored on the exception.
    print(f"Failed: needed {e.amount}, only had {e.balance}")
except AccountFrozenError as e:
    print(f"Failed: {e}")
except BankError as e:
    # Catches any OTHER BankError subclass we didn't handle explicitly.
    print(f"Unexpected bank error: {e}")

```

7. Common Pitfalls (Interview Favorites)

- Using a bare 'except:' -- silently swallows ALL errors, including typos in your own code, making bugs nearly impossible to find.
- Catching Exception too early and too broadly -- hides real bugs instead of surfacing them.
- Forgetting that 'finally' always runs -- even if the try or except block contains a return statement.
- Not using 'raise ... from e' when wrapping an exception -- you lose the original traceback, making debugging much harder.
- Putting too much code inside 'try' -- keep the try block focused on just the risky line(s) so you know exactly what failed.
- Using exceptions for normal control flow -- e.g. checking 'if key in dict' is usually clearer than catching KeyError everywhere.

8. Full Runnable Example

A complete script combining try/except/else/finally, raising, re-raising, and a custom exception -- ready to paste and run.

```
# error_handling_demo.py
# Complete, runnable demonstration of Python error handling.

class InvalidAgeError(Exception):
    """Raised when an age value fails validation."""
    def __init__(self, age):
        self.age = age
        super().__init__(f"Invalid age: {age}")

def validate_age(age):
    """Raise InvalidAgeError if age is not a sane value."""
    if not isinstance(age, int):
        raise TypeError(f"Age must be an int, got {type(age).__name__}")
    if age < 0 or age > 130:
        raise InvalidAgeError(age)
    return age

def safe_divide(a, b):
    """Divide two numbers, returning None on division by zero."""
    try:
        result = a / b
    except ZeroDivisionError:
        print(f"Cannot divide {a} by zero.")
        return None
    else:
        # Only runs if the division succeeded.
        print(f"{a} / {b} = {result}")
        return result
    finally:
        # Always runs -- useful for logging that the attempt happened.
        print(f"Finished attempting {a} / {b}")
```

```

def main():
    # 1) Custom exception in action
    for test_age in [25, -5, 200]:
        try:
            validate_age(test_age)
            print(f"Age {test_age} is valid.")
        except InvalidAgeError as e:
            print(f"Rejected: {e}")
        except TypeError as e:
            print(f"Rejected: {e}")

    print("-" * 30)

    # 2) else/finally in action
    safe_divide(10, 2)
    safe_divide(10, 0)

if __name__ == "__main__":
    main()

# Expected output:
# Age 25 is valid.
# Rejected: Invalid age: -5
# Rejected: Invalid age: 200
# -----
# 10 / 2 = 5.0
# Finished attempting 10 / 2
# Cannot divide 10 by zero.
# Finished attempting 10 / 0

```

Quick recap: 'try' wraps risky code, 'except' handles specific failures, 'else' runs only on success, 'finally' always runs. Use 'raise' to signal your own errors, 'raise ... from e' to preserve context, and custom exception classes to make error handling self-documenting.